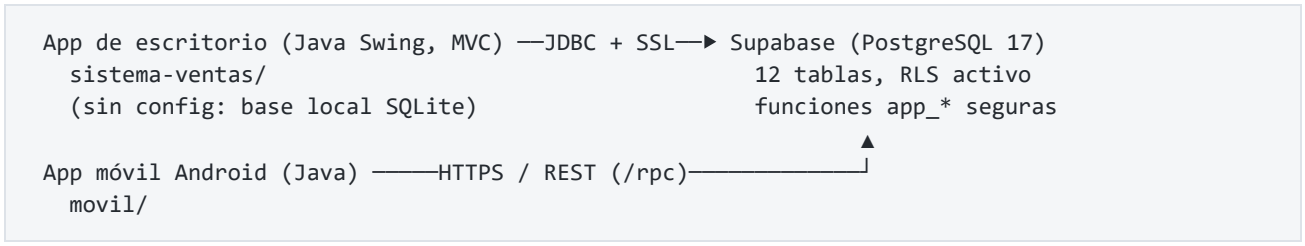
Manual Técnico — Sistema de Ventas

Universidad Mariano Gálvez · Programación II

Autor: Josue Zetino · Repositorio: https://github.com/jzetinob/sistema-ventas

1. Visión general

El sistema tiene tres piezas que comparten los mismos datos:



Pieza	Tecnología	Carpeta
Escritorio	Java 25, Swing (MDI con `JDesktopPane`), Maven, NetBeans	`sistema-ventas/`
Base de datos	PostgreSQL 17 en Supabase (nube) o SQLite (local, sin configuración)	`sistema-ventas/src/main/resources/bd/`
Móvil	Android (Java 17; minSdk 24, compileSdk 37, targetSdk 34). Solo usa lo que trae Android, más la librería oficial SwipeRefreshLayout. Se compila con Gradle	`movil/`
Documentación	Markdown	`docs/`, `tareas/`

2. Requisitos de desarrollo

Herramienta	Versión
JDK	25 (`maven.compiler.release 25`)
Apache NetBeans	22 o superior (probado con 25)
Maven	el que incluye NetBeans
Android Studio	2026.1 o superior (SDK de Android, API 37)
Gradle / Android Gradle Plugin	9.8 / 9.4.1 (incluidos: `movil/gradlew`)
Cuenta de Supabase	plan Free

Dependencias Maven (`sistema-ventas/pom.xml`):

Dependencia	Uso
`org.xerial:sqlite-jdbc:3.47.1.0`	Base local SQLite
`org.postgresql:postgresql:42.7.4`	Base en la nube
`org.mindrot:jbcrypt:0.4`	Hash de contraseñas (bcrypt)
`com.formdev:flatlaf:3.7.2`	Apariencia moderna de Swing con tema claro y oscuro

3. Arquitectura de la app de escritorio

Patrón **MVC** en el paquete `com.josue.ventas` :

Paquete	Contenido
`modelo`	Entidades: `Persona` (abstracta) → `Cliente` , `Empleado` , `Proveedor` ; `Cliente` + `Telefono` , `Producto` , `Categoria` , `Factura` + `FacturaDetalle` , `Compra` + `CompraDetalle` , `Usuario`
`dao`	Interfaces DAO y su implementación JDBC (`…DAOSQLite` , que funcionan también con PostgreSQL), `ConexionBD` , `CopiadorBD` , `MigradorDatos` (CSV antiguos)
`controlador`	Un controller por entidad, `Sesion` (usuario conectado), `Contrasenas` (hash), `RespaldoController`
`vista`	Formularios (`JInternalFrame`), `FrmCatalogo` (base abstracta), `PanelTelefonos` (lista de teléfonos del cliente), `DlgLogin` , `FrmResumen` + `GraficoBarrasPanel` / `GraficoDonaPanel` (gráficos con `Graphics2D`), `BarraBusqueda` , `FiltroTabla` , `ReporteHtml` , `Reportes` , `CampoBusqueda`

Flujo de una operación: **vista** → **controlador** → **DAO** → **base de datos**. Las vistas nunca escriben SQL.

3.1 Patrones de diseño aplicados

Patrón	Dónde
MVC	Separación `vista` / `controlador` / `modelo` + `dao`
Singleton	`ConexionBD` , cada `…DAOSQLite` , `Sesion` , `ApiSupabase` (móvil)
DAO	Interfaces `ClienteDAO` , `ProductoDAO` … con implementación intercambiable (CSV → SQLite/PostgreSQL) sin tocar las vistas
Template Method	`FrmCatalogo` arma la pantalla y define los pasos abstractos `guardar()` , `actualizar()` , `eliminar()` … que implementan `FrmCategorias` , `FrmProveedores` , `FrmUsuarios`
Herencia / polimorfismo	`Persona` abstracta; `Empleado` y `Proveedor` sobrescriben `mostrarInformacion()` ; `ActividadBase` en Android

Patrón	Dónde
Composición	`Factura` ◆— `FacturaDetalle`, `Compra` ◆— `CompraDetalle`, `Cliente` ◆— `Telefono`

3.2 Selección del motor de base de datos

`ConexionBD` lee `sistema-ventas/config/bd.properties` al arrancar:

- **No existe** → SQLite en `datos/sistema_ventas.db` (crea el archivo y las tablas).
- `bd.motor=postgresql` → se conecta a Supabase y ejecuta `bd/esquema-postgresql.sql`, que se puede correr siempre porque es idempotente.

```
bd.motor=postgresql
bd.url=jdbc:postgresql://aws-0-us-east-1.pooler.supabase.com:5432/postgres
bd.usuario=postgres.<referencia-del-proyecto>
bd.clave=<contraseña de la base de datos>
```

> `config/bd.properties` contiene la contraseña: está en `.gitignore` y **nunca** se sube a GitHub. En el repositorio solo está `bd.properties.ejemplo`.

Los DAO usan SQL estándar que funciona en ambos motores. Lo único específico de cada motor está en `ConexionBD`: la creación de tablas, las migraciones y el respaldo.

Primera conexión a la nube: si la base de Supabase está vacía y existe `datos/sistema_ventas.db` con datos, `CopiadorBD` los sube automáticamente. Conserva los `id` y ajusta las secuencias.

Migraciones: al crear la tabla `telefonos`, el script de PostgreSQL copia el teléfono que ya tenía cada cliente como tipo `PERSONAL` (`INSERT … SELECT … WHERE NOT EXISTS`, se puede repetir sin duplicar). En SQLite, las bases creadas con versiones anteriores se actualizan solas (`agregarColumnaSiFalta`): `productos.existencia`, `productos.categoria_id`, `factura_detalles.producto_id`. `MigradorDatos` importa una sola vez los CSV de las primeras versiones (`clientes.csv`, `productos.csv`, `facturas.csv`, `detalles.csv`) si la base está vacía. Los DAO CSV (`…DAOCsv`) quedaron en el código como referencia del entregable de listas y archivos.

3.3 Detalles de implementación

- **Ventana MDI:** `FrmPrincipal` contiene un `JDesktopPane`, y cada formulario es un `JInternalFrame`. Si una ventana ya está abierta (`isDisplayable()`), se trae al frente en lugar de crear otra. El menú *Ventana* acomoda en cascada o mosaico, y minimiza o restaura todas.
- **Correlativos (FAC-0001, COM-0001):** se calculan como el mayor número existente más uno, consultando la tabla. No hay un contador aparte, así que no se repiten al reiniciar y no se "queman" números al cerrar un formulario sin guardar.
- **Impresión:** `TicketFactura` implementa `java.awt.print.Printable` y dibuja el ticket con `Graphics2D`. `FrmVistaPreviaFactura` usa el mismo método `pintar(...)` para la vista previa y

abre el diálogo de impresión del sistema (`PrinterJob.printDialog()`). Se imprime desde la factura en curso y desde el detalle de una factura guardada, sin librerías externas.

- **Buscador con autocompletado (`CampoBusqueda`):**
  - Es un `JTextField` con un menú emergente y una `JList` . Filtra por "contiene", sin distinguir mayúsculas; se navega con ↑/↓, Enter elige y Escape cierra.
  - Vuelve a leer su lista (`setFuente`) cada vez que recibe el foco, así aparecen los registros creados en otra ventana o desde el celular.
  - Si nada coincide, muestra "(sin coincidencias)".
  - Avisa a sus oyentes (`addActionListener`) al elegir un elemento. Es el patrón Observer.
- **Apariencia y tema por hora (`Tema`):**
  - Usa FlatLaf (`FlatLightLaf` / `FlatDarkLaf`), con el color de acento de la app móvil y esquinas redondeadas.
  - En modo automático, un `javax.swing.Timer` revisa la hora cada minuto: claro de 06:00 a 17:59, oscuro de 18:00 a 05:59. `FlatLaf.updateUI()` repinta todas las ventanas abiertas.
  - El modo (Automático, Claro u Oscuro) se elige en *Ventana → Tema* y se guarda con `java.util.prefs.Preferences` .
  - Los componentes pintados a mano (gráficos y `PanelTarjeta`) piden sus colores a `Tema` en cada pintado. `FrmResumen` se registra con `Tema.alCambiar(...)` para refrescarse cuando cambia el tema.
- **Resumen con gráficos (`FrmResumen`):** `ResumenController` toma una "foto" de las facturas y los productos (a través de sus controllers, así funciona con los dos motores) y calcula los indicadores, las ventas de cada uno de los últimos 7 días (con ceros), el top 5 del mes y la existencia baja. `GraficoBarrasPanel` y `GraficoDonaPanel` sobrescriben `paintComponent()` y dibujan con `Graphics2D` (`fillRoundRect` , `Arc2D`). Es la misma idea que `onDraw()` con `Canvas` en Android.
- **Búsqueda en tablas (`FiltroTabla` , `BarraBusqueda`):** usan `TableRowSorter` y `RowFilter` . Cuando la tabla está filtrada, la fila seleccionada se traduce con `convertRowIndexToModel` .
- **Varios teléfonos por cliente (relación 1 a N):**
  - `Cliente` tiene una `List<Telefono>` ; cada `Telefono` guarda `clienteId` , `numero` y `tipo` (enum `Telefono.Tipo` : `PERSONAL` , `TRABAJO` , `CASA` , `OTRO`). En la base es la tabla `telefonos` , sin campos fijos `telefono1` , `telefono2` …
  - `PanelTelefonos` (vista) agrega, modifica y quita teléfonos de una lista en memoria. `FrmClientes` lo contiene, y al seleccionar un cliente carga sus teléfonos desde la base.
  - `ClienteController.Guardar` / `Actualizar` validan los teléfonos (8 a 15 dígitos, con tipo, sin repetirse), copian el primero a `clientes.telefono` (*teléfono principal*, que usan la app móvil y los reportes) y guardan el cliente con `ClienteDAO` y sus teléfonos con `TelefonoDAO` . Devuelven `null` o el mensaje de error.
  - `TelefonoDAOSQLite.reemplazar` guarda la lista en una transacción: `DELETE` de los teléfonos del cliente e `INSERT` de los de la lista. Es la misma estrategia que `FacturaDAOSQLite.actualizar` usa con los detalles.

4. Base de datos

4.1 Tablas

Hay 11 tablas del sistema, más `sesiones_app` para el móvil (solo en PostgreSQL). El diagrama completo está en `diagramas/diagrama-entidad-relacion.md`.

Tabla	Descripción	Llaves foráneas
`categorias`	Clasificación de productos	—
`productos`	Catálogo, precio y existencia	`categoria_id` → categorias (SET NULL)
`clientes`	NIT único; `telefono` guarda el teléfono principal	—
`telefonos`	Teléfonos de cada cliente: número y tipo (`CHECK`)	`cliente_id` → clientes (CASCADE)
`empleados`	Código único	—
`proveedores`	NIT único	—
`usuarios`	Usuario, hash de contraseña, rol, activo	—
`facturas`	Encabezado de venta	—
`factura_detalles`	Líneas de la factura	`factura_id` (CASCADE), `producto_id` (SET NULL)
`compras`	Encabezado de compra	`proveedor_id` (RESTRICT), `usuario_id` (SET NULL)
`compra_detalles`	Líneas de la compra	`compra_id` (CASCADE), `producto_id` (RESTRICT)
`sesiones_app`	Tokens de la app móvil (8 h)	`usuario_id` (CASCADE)

En SQLite, las llaves foráneas se activan en cada conexión con `PRAGMA foreign_keys = ON`.

4.2 Transacciones

Estas operaciones se ejecutan completas o no se ejecutan (`setAutoCommit(false)` + `commit` / `rollback`):

- **Guardar factura:** inserta el encabezado y los detalles, y descuenta la existencia con `UPDATE … WHERE existencia >= ?`. Si un producto no alcanza, se revierte todo.
- **Eliminar factura:** devuelve la existencia y borra la factura (los detalles se borran en cascada).
- **Guardar compra:** inserta el encabezado y los detalles, y suma la existencia.
- **Anular compra:** resta la existencia (se rechaza si ya no alcanza) y borra la compra.
- **Guardar los teléfonos de un cliente:** borra los anteriores e inserta los de la lista.

5. Seguridad

Medida	Implementación
Inyección SQL	Todo dato del usuario viaja en `PreparedStatement`. Los únicos SQL armados con texto usan nombres de tablas y columnas fijos del código
Contraseñas	bcrypt, costo 10 (`Contrasenas`). Los hash antiguos PBKDF2-SHA256 (120 000 iteraciones) siguen siendo válidos y se convierten a bcrypt en el siguiente inicio de sesión
Inicio de sesión	El mismo mensaje para usuario inexistente y contraseña incorrecta; cierre tras 3 intentos; `char[]` limpiado después de usarse
Roles	Administrador / Vendedor; el menú se ajusta al rol (`FrmPrincipal.aplicarSesion`)
Reglas de negocio	No es posible eliminarse a uno mismo; siempre queda al menos un administrador activo
Reportes	Todo dato se escapa (`ReporteHtml.escapar`) antes de escribirse en HTML
Búsqueda	`Pattern.quote`: lo que escribe el usuario no se interpreta como expresión regular
Credenciales	La contraseña de la base solo está en `config/bd.properties` (ignorado por Git)
API pública (Supabase)	**RLS activo en las 12 tablas y sin políticas**, y permisos de tablas revocados a `anon` / `authenticated`. La API no puede leer ni escribir tablas. Solo puede ejecutar las funciones `app_*`
Funciones móviles	`SECURITY DEFINER` con `search_path` fijo; todas validan el token (`app_usuario_de`); el login espera 1 s tras un fallo; los tokens vencen a las 8 h y `app_logout` los elimina

5.1 API de la app móvil

Todas se llaman con `POST https://<proyecto>.supabase.co/rest/v1/rpc/<función>`, el encabezado `apikey: <clave publishable>` y un cuerpo JSON.

Función	Parámetros	Devuelve
`app_login`	`p_usuario`, `p_clave`	`{token, nombre, rol}`
`app_logout`	`p_token`	—
`app_productos`	`p_token`, `p_buscar`	`[{codigo, nombre, categoria, precio, existencia}]`
`app_clientes`	`p_token`, `p_buscar`	`[{nit, nombre, direccion, telefono}]`
`app_registrar_cliente`	`p_token`, `p_nit`, `p_nombre`, `p_direccion`, `p_telefono`	`{id, nit, nombre}`
`app_ventas`	`p_token`, `p_desde`, `p_hasta` (aaaa-mm-dd)	`[{numero_factura, fecha, cliente, total}]`

Función	Parámetros	Devuelve
`app_detalle_venta`	`p_token`, `p_numero_factura`	`[{producto, cantidad, precio, subtotal}]`
`app_resumen`	`p_token`, `p_hoy` (fecha local del teléfono), `p_existencia_baja` (5)	`{ventas_hoy, facturas_hoy, ventas_mes, ventas_7_dias[], top_productos[], existencia_baja[]}`
`app_registrar_factura`	`p_token`, `p_nit`, `p_cliente`, `p_fecha`, `p_detalles` (`[{codigo, cantidad}]`)	`{numero_factura, total}`

Los errores llegan con código HTTP 4xx y `{"message": "..."}`. El mensaje ya está en español y la app lo muestra tal cual.

6. App móvil (`movil/`)

Paquete	Clases
`datos`	`ConfigSupabase` (URL y clave publishable), `ApiSupabase` (singleton y fachada: llamadas HTTP en un hilo aparte con `ExecutorService`, respuesta en el hilo principal con `Handler`), `Sesion` (token en `SharedPreferences`; nunca guarda la contraseña), `CacheCatalogo` (copia del catálogo para usar sin conexión)
`modelo`	`Producto`, `Cliente`, `Venta`, `LineaVenta` (detalle y carrito), `Resumen` (con `VentaDia` y `ProductoVendido`)
`vista`	`LoginActivity`; `MenuActivity` (inicio con indicadores y menú en mosaico); `ResumenActivity`; `NuevaVentaActivity`; `DetalleVentaActivity`; `ProductosActivity`; `ClientesActivity`; `NuevoClienteActivity`; `VentasActivity`; `ActividadBase` (abstracta, exige sesión); `AdaptadorFilas<T>` (tarjetas, genérica); `DialogoBusqueda<T>` (elegir cliente o producto, genérica); `GraficoBarras` y `GraficoDona` (extienden `View` y dibujan con `Canvas` en `onDraw()`); `Formatos`

HTTP (`HttpURLConnection`), JSON (`org.json`) y gráficos (`Canvas`) usan solo lo que trae Android. La **única dependencia** es `androidx.swiperefreshlayout`, el componente oficial para *deslizar para actualizar*.

Detalles de la app móvil:

Tema	Cómo funciona
Gráficos	`GraficoBarras` escala cada barra al máximo de la semana y escribe su valor encima; destaca el día de hoy. `GraficoDona` dibuja un arco por producto, proporcional a sus unidades (`drawArc`), y pone el total en el centro. Los colores salen de `colores.xml`, así que también cambian en modo oscuro

Tema	Cómo funciona
Factura móvil	El teléfono envía solo el código y la cantidad de cada producto. `app_registrar_factura` bloquea la tabla para asignar el número correlativo, toma el precio del catálogo, bloquea cada producto (`SELECT … FOR UPDATE`), valida la existencia y la descuenta. Todo es una transacción: si algo falla, no se guarda nada
Sin conexión	Cada consulta completa del catálogo se guarda en `SharedPreferences` (`CacheCatalogo`). Si no hay red, Productos muestra esa copia filtrada, con un aviso y su fecha. Se borra al cerrar sesión
Tema por hora	`values-night/colores.xml` y `values-night/estilos.xml` redefinen la paleta y el tema. `TemaPorHora` decide según la hora: claro de 06:00 a 17:59, oscuro de 18:00 a 05:59. `ActividadTema` , la clase padre de todas las pantallas, crea en `attachBaseContext` una copia de la configuración con el "modo noche" que toca, y Android toma los recursos de `values/` o `values-night/` . Cada minuto revisa la hora y, si cambió, recrea la pantalla (`recreate()`)
Fechas	Se usa `Calendar` y `SimpleDateFormat` porque `java.time` requiere Android 8 y la app funciona desde Android 7. El teléfono envía su fecha local (`p_hoy`) porque el servidor trabaja en UTC

Compilar: abra la carpeta `movil/` en Android Studio y use *Build → Build APK(s)*, o desde la terminal ejecute `gradlew assembleDebug` con `JAVA_HOME` apuntando al JDK que trae Android Studio (carpeta `jbr`). El APK queda en `movil/app/build/outputs/apk/debug/` , y la copia lista para instalar está en `entregables/VentasMovil.apk` .

targetSdk 34: se deja así a propósito. Desde el API 35, Android obliga el modo de pantalla completa (*edge-to-edge*), que taparía la barra de título en este diseño con vistas clásicas.

7. Instalación y puesta en marcha

7.1 Escritorio (desarrollo)

1. Clone el repositorio y abra `sistema-ventas` en NetBeans (proyecto Maven).
2. **Modo local:** ejecute (F6). No se necesita nada más.
3. **Modo nube:** copie `config/bd.properties.ejemplo` como `config/bd.properties` y complete los datos del *Session pooler* de Supabase. Al ejecutar se crean las tablas y funciones.
4. En el primer uso, el sistema pide crear el administrador.

7.2 Crear el proyecto en Supabase (una vez)

1. Cree una cuenta en supabase.com, una organización del plan **Free** y un proyecto (región `us-east-1`).
2. **Connect → Session pooler:** anote el host y el usuario para `bd.properties` .
3. **Project Settings → API Keys:** copie la *Project URL* y la *publishable key* a `movil/.../ConfigSupabase.java` .
4. Ejecute la app de escritorio una vez: crea todo el esquema.

> El plan Free **pausa el proyecto tras 7 días sin actividad**. Se reactiva desde el panel de Supabase con **Restore project**.

8. Respaldo y restauración

- **Respaldo:** *Administración → Respaldar base de datos* genera un archivo `.db` (SQLite) con todas las tablas. Funciona igual con la base local y con la nube; desde la nube se copian las tablas con `CopiadorBD`.
- **Restaurar en local:** cierre el sistema, reemplace `datos/sistema_ventas.db` por el respaldo y quite `config/bd.properties`.
- **Restaurar en la nube:** con la base de la nube vacía, coloque el respaldo como `datos/sistema_ventas.db` y abra el sistema en modo nube. La importación automática sube los datos.

9. Pruebas realizadas

Se ejecutaron pruebas automatizadas contra bases temporales (SQLite) y contra el proyecto real de Supabase. Al terminar, se borraron los datos de prueba.

Área	Qué se verificó
Esquema	11 tablas (+1 en la nube), llaves foráneas activas, RLS en las 12 tablas
Teléfonos de clientes	Guardar con dos teléfonos, consultar de nuevo, modificar y agregar un tercero, rechazo de números inválidos y repetidos, borrado en cascada al eliminar el cliente y migración de los teléfonos existentes. Probado con SQLite y contra la nube
Migración	Una base de una versión anterior se actualiza sin perder datos
Usuarios	Validaciones, contraseñas hasheadas, intento de inyección SQL rechazado, conversión de PBKDF2 a bcrypt, reglas de administrador
Inventario	La compra suma, la factura descuenta, una venta sin existencia se rechaza, eliminar o anular devuelve la existencia
Integridad	No se borran productos ni proveedores con compras; las categorías pasan a SET NULL; los detalles se borran en cascada
Búsquedas y reportes	Filtro literal, selección correcta con la tabla filtrada, escape HTML
Respaldo	Local (`VACUUM INTO`) y desde la nube a `.db`
API móvil	Login correcto e incorrecto, token falso o cerrado rechazado, consultas, alta de cliente visible en el escritorio, **lectura directa de tablas bloqueada (401)**

Área	Qué se verificó
Resumen y factura móvil	`app_resumen` (7 días con ceros, top, existencia baja) y `app_registrar_factura` (número, precio de la base, descuento de existencia, rechazo por existencia, NIT, repetidos y productos inexistentes). Se probaron **contra la nube dentro de una transacción revertida**, sin tocar los datos reales. Resumen de escritorio: cálculos verificados con una base temporal